COMP5339 期末考场速查

W1-W10 · 一张 A4 双面 · 中英术语与答题模板

W1 Data Engineering / Pipeline

• Data engineering：把 raw data 变成高质量、一致、可供 analytics/ML 使用的信息。
• 生命周期：generation/source -> acquisition/ingestion -> storage -> transformation -> serving；横切 security、DataOps、architecture、orchestration。
• Source=数据起点；sink=输出目的地；pipeline=按依赖连接的处理步骤。Batch处理有界批次；stream持续处理无界事件。

W1 Unix 高频

• pipe | 把左命令 stdout 交给右命令；>覆盖，>>追加；grep筛行；wc -l计行；head/tail查看；cut选列；sort排序；uniq只去除相邻重复。

```
sort names.txt | uniq
grep -i 'error' log.txt | wc -l
cut -d',' -f1 file.csv | sort | uniq -c
```

W2 Acquisition / Ingestion

• 优先官方 API：结构化、稳定、面向程序；无 API 才 scraping。Pull=消费者主动定期取；Push=事件发生即发送，适合低延迟。
• Batch适合周期历史数据；stream适合连续实时。CDC 捕捉 insert/update/delete 变化。
• 工程判断：volume、velocity、variety、刷新频率、权限、rate limit、失败重试、幂等。

W2 Cleaning / Quality

• Profiling先看 schema、count、NULL、distinct、range、format、duplicates、outliers。
• Validation=检查约束，不等于修复：类型/日期格式/范围/唯一性/引用完整性。
• Cleaning：统一单位/日期/类别、处理缺失、去重、标准化；保留 raw 与审计记录。
• Missing：删、填补或标记取决于机制/任务；异常值先验证，不可盲删。

W3 Relational DB

• 表=rows+columns；schema定义类型/约束。PK唯一非空标识行；FK引用另一表 PK，保证 referential integrity。
• 1NF原子值；2NF消除对复合键部分依赖；3NF消除非键对非键传递依赖。规范化减少冗余，反规范化换取读性能。
• INNER只匹配；LEFT保留左表；NULL 用 IS NULL；COUNT(*)含所有行，COUNT(col)忽略NULL。

SQL 模板

• WHERE筛行；GROUP BY分组；HAVING筛聚合组；ORDER BY排序。非聚合 SELECT 列必须在 GROUP BY。
• JOIN 先明确 grain 和 key，避免 many-to-many 行数爆炸；先查每边 key 是否唯一。

```
SELECT a.k, COUNT(*) n, AVG(b.v) av
FROM a LEFT JOIN b ON a.id=b.a_id
WHERE a.ts>=:s AND a.ts<:e
GROUP BY a.k HAVING COUNT(*)>1
ORDER BY av DESC;
```

Warehouse / Lake / ETL

• OLTP：频繁小事务、当前细节、规范化；OLAP：扫描聚合、历史分析、星型模型。
• Fact表=度量+维度FK，grain必须先定义；Dimension=描述属性；star简单，snowflake更规范。
• ETL先转换再入仓；ELT先加载 raw 再在目标中转换。Data warehouse=schema-on-write curated；Data lake=schema-on-read保存raw/多类型；lakehouse试图兼具。
• Data mart服务部门；staging是临时加工区。

W4 Web API / HTTP

• API优于 scraping：结构稳定、机器接口、常有文档。GET读取，POST提交；参数可在URL/query/body；响应常 JSON/XML。
• 检查 status code：2xx成功，4xx客户端/认证/限流，5xx服务端；处理 timeout、retry+backoff、pagination、rate limit。
• 先检查 content-type 与 schema，再解析；认证密钥不写入代码仓库。

```
r=requests.get(url,params=p,timeout=30)
r.raise_for_status(); data=r.json()
```

W4 Scraping

• HTML由 DOM tree 组成；CSS selector/XPath定位元素。先遵守 robots/条款/频率。
• BeautifulSoup：select/select_one、get_text(strip=True)、get('href')；动态JS页面可能需浏览器或找底层API。
• 选择稳定语义属性，避免脆弱绝对路径；记录抓取时间和来源URL。

```
soup=BeautifulSoup(html,'html.parser')
for x in soup.select('table tbody tr'): ...
```

W5 Semi-structured / JSON

• JSON object={}键值；array=[]有序列表；值可嵌套。灵活 schema 允许缺失/额外字段与异构类型。
• 解析嵌套时逐层访问并处理缺失；normalize/flatten 要避免数组导致行爆炸。
• 字段名表达 schema，日期/时间应作为值：{'values':[{'date':'2026-01-01','sales':100}]}，便于范围查询和索引。

W5 NoSQL / MongoDB

• Key-value：按key超快；Document(MongoDB)：BSON文档+嵌套；Wide-column：row key/column family；Graph：node/edge关系遍历。
• MongoDB 存 documents，支持 nested structure、灵活schema，不是SQL主语言也不是graph DB。
• Embed适合一起读取且有界的一对少；Reference适合共享、增长、独立更新。索引提速读但增加写/空间。

```
db.col.find({status:'A',score:{$gte:80}})
db.col.createIndex({status:1})
```

COMP5339 期末考场速查

W1-W10 · 一张 A4 双面 · 中英术语与答题模板

W6 Spatial 基础

• Geometry：Point/LineString/Polygon/MultiPolygon。Spatial join 如 point-in-polygon；predicate：intersects/within/contains；distance/nearest。
• CRS定义坐标含义；EPSG:4326是经纬度。经纬顺序常 lon,lat。不同 CRS 必须先 transform，不能只改标签。
• Geometry常平面单位；Geography/投影可用于地球距离。经纬度直接用欧氏距离会失真。

```
ST_Point(lon,lat)
ST_Within(point, polygon)
ST_Intersects(a,b)
ST_Distance(a,b)
```

W6 Index / 格式

• Spatial index(R-tree等)用 bounding boxes
先过滤候选，减少精确几何计算；加速查询但不转换 CRS，也不限 point。
• Shapefile 是多文件集合(.shp/.shx/.dbf/.prj)；GeoJSON易交换；WKT是文本几何表示。
• 空间连接前验证 geometry有效性、CRS、范围、空值与边界语义。

□ 正确选项：空间索引可加速查询、减少需精算的geometry；不会自动坐标转换。

W7 Temporal 超压缩

• Valid=现实有效；Transaction=数据库知道；Bitemporal=两者。Period用[start,end)；overlap：a.s<b.e AND b.s<a.e。
• WHERE筛行 -> GROUP BY -> aggregate -> HAVING筛组。array_agg 内 ORDER BY；Spark collect_list 不保序。
• PostgreSQL range：&& overlap，@> contains，GiST索引。

W8 Unstructured 超压缩

• 有label监督：分类/回归；无label无监督：聚类/topic。split后只在train fit，防 leakage。
• BoW词频；TF-IDF=tf*log(N/df)；文本向量常cosine。Word2Vec静态，BERT上下文理解，GPT next-token生成。
• 图像HxWxC；检查尺寸/通道/范围；metadata(EXIF)可结构化但可能缺失/错误。

W9 Broker / Semantics

• Pub/Sub：producer->topic->broker->consumer。MQTT +一层/#多层；QoS0最多一次、1至少一次、2单跳恰好一次。
• Kafka=分区持久日志；partition内有序；offset为位置；同group一个partition同一时刻给一个consumer；key保同key顺序。
• 端到端 exactly-once 需要source+checkpoint+transactional/idempotent sink。

W9 Time / Window / State

• Event=发生，Ingestion=入平台，Processing=执行；乱序业务用event time。
• Tumbling不重叠；Sliding可重叠；Session按空闲间隔。Watermark≈max event time-容忍乱序，用于关窗/清state，不是绝对保证。
• map/filter无状态；count/dedup/window/join有状态。stream-stream join必须key+时间界。

W10 Storage / Distribution

• 行存适合整行写/点查；列存/Parquet适合列扫描聚合；wide-column不是分析型列存。
• Range partition利范围但易热点；hash均匀；skew使全作业等最慢分区。Replication提可用，不是backup。
• CAP：网络分区时C与A取舍。Mongo：mongos路由/config元数据/shards数据。
• HDFS：NameNode元数据，DataNode blocks。MapReduce：map->shuffle/sort->reduce。

W10 Spark

• Driver建DAG，executors跑partitions；transformation惰性，action触发。宽依赖常shuffle。
• select/filter/groupBy/agg/join；temp view 后 spark.sql。避免collect/toPandas大数据，优先内建函数。
• Flink原生逐事件/event-time/state强；Spark micro-batch、统一批流与SQL生态强。

```
df.filter(F.col('x')>0)\
.groupBy('k').agg(F.avg('v'))
```

选择题触发词 -> 直接选

• real-time/as soon as occurs -> push/stream；structured programmatic access -> Web API。
• raw now, unknown future use, transform later -> data lake/ELT。
• DOM + CSS selectors -> scraping；documents + nested -> MongoDB。
• current row overwritten -> need temporal history；late/out-of-order -> event time + watermark。
• scan few columns/aggregate -> columnar/Parquet；hotspot -> bad partition key/skew。
• same key ordering/replay -> Kafka partition；IoT轻量 -> MQTT。

代码题自检

• 1 写清输入 grain/key/time unit。2 用半开时间范围。3 JOIN前检查唯一性。4 WHERE与HAVING分清。5 聚合后核对count/min/max。
• JSON先看 object/array nesting；SQL处理NULL用IS NULL/COALESCE；空间先CRS；流式写window/time/watermark。

常见绝对化陷阱

• ‘always/only/automatically/exclusively’ 多为错：NoSQL不代表无schema；index不改CRS；QoS2不保证整管道恰好一次；更多partition/机器不必然更快。
• 相似度越大越近，距离越小越近；BETWEEN含两端；COUNT(col)不计NULL；LEFT JOIN后在WHERE筛右表可能变INNER效果。

10分钟读题法

• 圈动词：define/compare/design/calculate/write SQL；圈约束：real-time、late、range、latest、all rows。
• 选择题先排除类别错误和绝对化陈述；多选逐项判真，不凭‘看起来像两个’。
• 简答用四句：定义 -> 机制 -> 为什么适合本题 -> 一个限制/陷阱。SQL先写FROM/JOIN和WHERE，再GROUP/HAVING/SELECT。